Modeling Real Estate Values: Insights from Nashville Housing Data

Course: MSIS 672 Machine Learning

Date: May 14, 2025

Final Semester Project

Group Members:

Abhishek Vinod Nair- a.yalamanchili001@umb.edu -MSBA

Aditya Yalamanchili- a.yalamanchili001@umb.edu -MSBA

Drishti Gambhir - drishti.gambhir001@umb.edu -MSBA

Lakshmanan Natesan – l.natesan001@umb.edu -MSBA

Pranav Pagadala -Pranav.pagadala001@umb.edu -MSBA

Praveen Garapati -p.garapati001@umb.edu-MSBA

Table of Contents:

S.NO	Section	Page Number
1	Introduction	3-4
2	Data Understanding and Preparation	5-8
3	Modeling	9-14
4	Recommendations	15-17
5	Conclusion	17
6	Appendices	18-24
7	References	25

1.Introduction

Background of the Research Project

The real estate sector has evolved into one of the most dynamic and economically crucial industries, with data-driven tactics increasingly affecting decision-making processes. Accurate property appraisal is important not only for homebuyers and sellers, but also for investors, developers, and government agencies. These stakeholders rely largely on accurate, reliable, and insightful information to determine property valuations, market trends, and profitable investment prospects.

Historically, property assessment was primarily handled using manual appraisals or basic statistical models. However, standard methodologies frequently fail to represent the complexities and volatility of real estate markets. A variety of interdependent factors influence housing costs, including structural attributes (square footage, number of beds), temporal characteristics (building year), and locational aspects (neighborhood quality, accessibility to amenities, school districts). These components interact in nonlinear and frequently unforeseen ways, making appraisal both difficult and error prone.

The rise of big data and the evolution of machine learning have made it possible to build smarter systems that improve upon traditional evaluation methods. By learning from past data, these models can uncover patterns that might not be obvious and provide more accurate, scalable predictions. In a rapidly growing city like Nashville—where housing needs, urban expansion, and population changes are always in motion—using a data-driven approach is essential. It helps city planners, developers, and buyers better understand housing trends and make informed decisions.

1.1 Project Objective

This project aims to develop a predictive model for housing prices in Nashville using machine learning techniques. Specifically, we will:

- Preprocess and clean the Nashville Housing dataset to ensure quality input for modeling.
- Perform exploration data analysis (EDA) to understand key variables and their relationships.
- Apply a series of regression-based machine learning models (e.g., linear regression, neural network, decision trees, random forest, and K-Means Clustering) to predict house prices.
- Evaluate and compare these models using relevant performance metrics (such as RMSE and MAE).
- Interpret feature importance and explain model outputs to increase transparency and trust.

- Optionally, develop visualizations or a dashboard to demonstrate how such a model could be integrated into a decision-support system (DSS).

To follow this process, we follow the **CRISP-DM** methodology (Cross-Industry Standard Process for Data Mining), which ensures a structured approach through the stages of business understanding, data understanding, data preparation, modeling, evaluation, and deployment.

1.2 Expected Contribution

The fundamental contribution of this study is to demonstrate how machine learning may improve the accuracy, speed, and reliability of real estate price projections. By deploying this model, we provide

- **Improved Decision-Making**: Real estate agents, buyers, and investors can use the model to assess fair market value, reducing risks of over- or underpricing.
- **Data-Driven Insights**: The model uncovers which features (e.g., location, size, year built) are most influential in determining house prices, offering strategic insights for construction, development, and renovation.
- **Scalability and Automation**: Compared to manual evaluations, the model may be scaled across huge datasets and geographic regions, producing consistent findings quickly.
- **Support for Policy and Planning:** Local governments and urban planners can utilize aggregated model outputs to track affordability trends and support evidence-based zoning and development plans.
- Ultimately, this project contributes to a more transparent, efficient, and intelligent real estate ecosystem in Nashville, with the potential to be adapted for broader use in other cities or regions.

2. Data Understanding and Preparation

2.1 Data Source

The dataset used in this project is the **Nashville Housing dataset**, which contains records of real estate transactions within the city of Nashville, Tennessee. It includes information on property characteristics, transaction dates, ownership, and location details. This data set provides a rich and varied set of features suitable for modeling real estate prices using machine learning.

The dataset includes the following key columns:

- **Parcel ID**: Unique identifier for each property.
- **Land Use**: Describes the type of property (e.g., residential, commercial).
- **Property Address**: The street address of the property.
- **Owner Address**: The mailing address of the property owner.
- **Sale Date**: Date when the property transaction occurred.
- **Sale Price**: The price at which the property was sold (target variable).
- **Legal Reference**: Reference to the legal record of the transaction.
- **Year Built**: The year the property was constructed.
- **Bedrooms/Bathrooms**: Number of rooms in the property.
- **Square Footage**: Total living area of the property.

This dataset was stored in a **relational format** and imported into a MySQL database for structured data cleaning prior to model training.

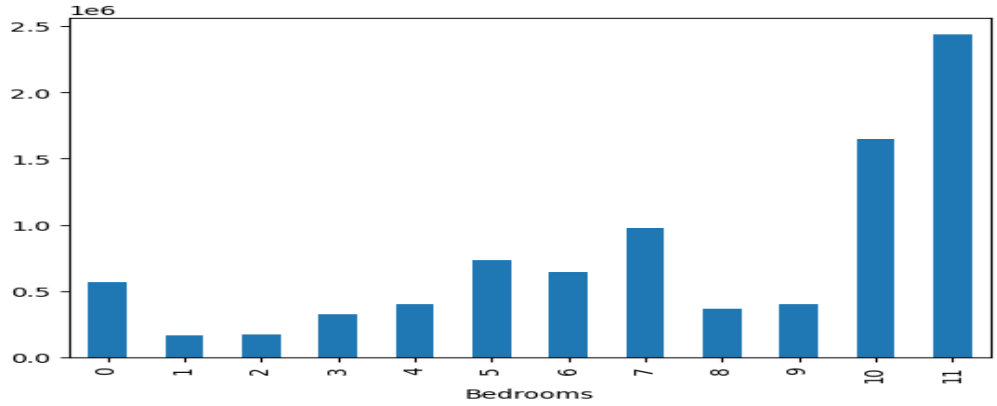

Figure 1: Bar Plot Representing Bedrooms with their mean Price

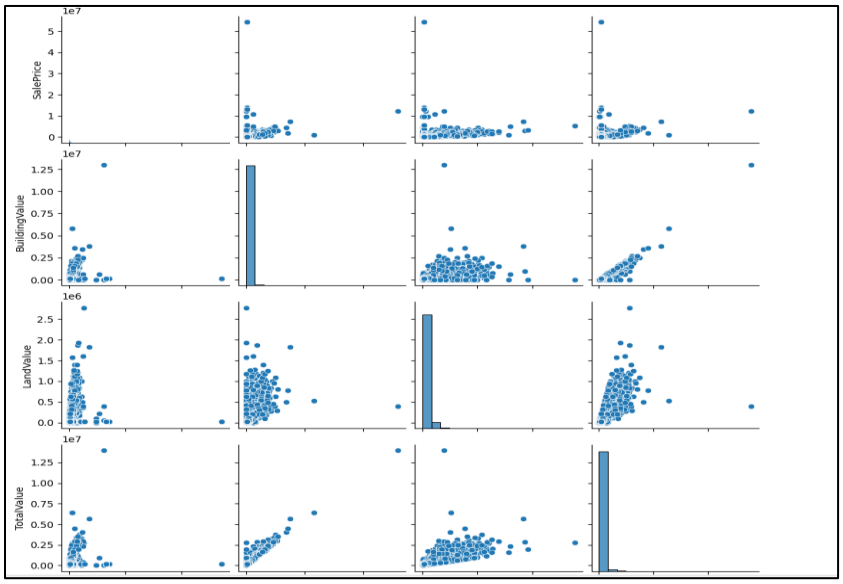

Figure 2: Pair Plot Representing Total value, land value, Building Value, Sale Price

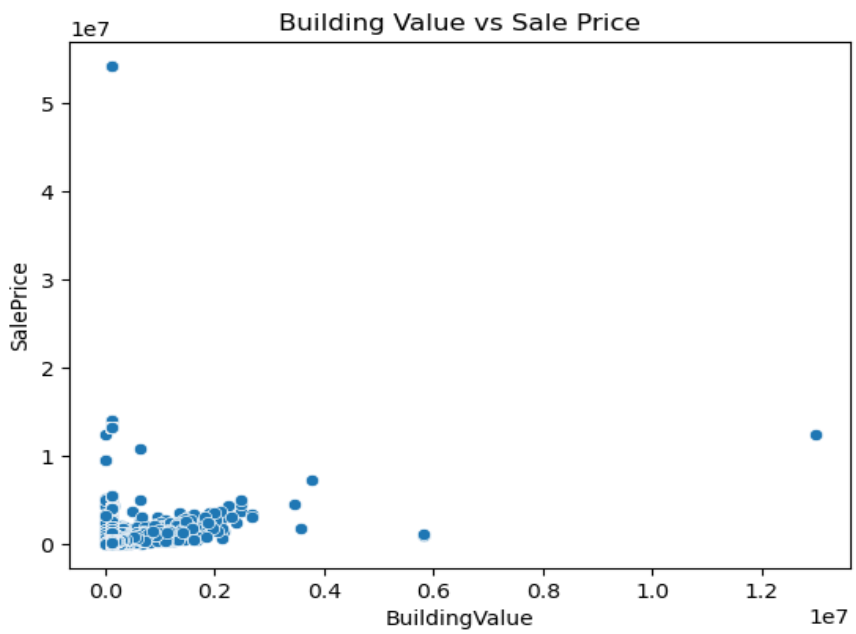

Figure 3: Scatter Plot Representing Building value and Sale Price

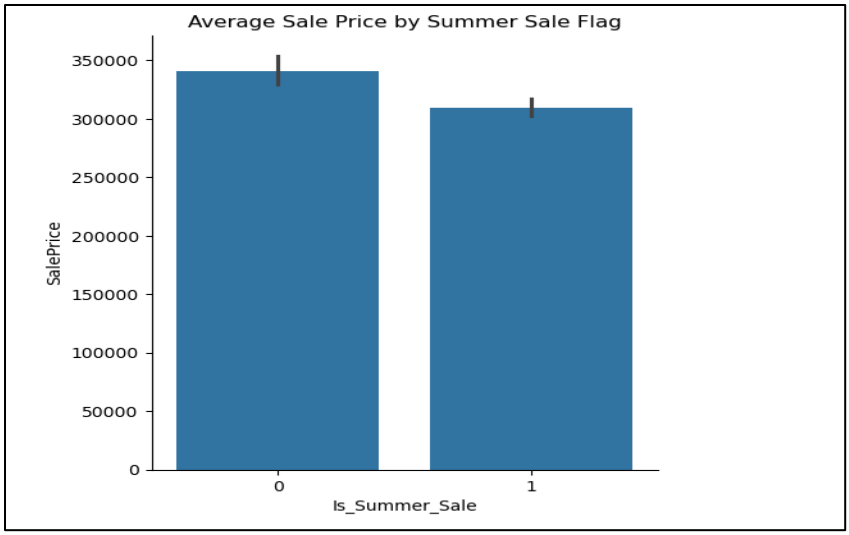

Figure 4: Average Sale Price by Summer flag

2.2 Data Cleaning and Processing in SQL

MySQL was used for the first data wrangling and transformation tasks because of its ability to handle big datasets and execute column-wise operations. The following are the key cleaning steps carried out.

1. Handling Missing Values

- Many entries in the Property Address column were found to be null.
- To handle this, the dataset was **ordered using the ROW_NUMBER ()** function (partitioned by Parcel ID or a similar identifier), and missing values were **populated with existing non-null values** from other rows with matching Parcel ID.

2. Data Type Conversion

- The Sale Date column was initially stored as a string. It was converted to a proper **DATE format** using SQL's STR_TO_DATE () function, making it usable for temporal analysis and sorting.

3. Splitting Compound Columns

- Both the Property Address and Owner Address fields contained multiple components in a single string, separated by commas (e.g., "123 Main St, Nashville, TN").
- These fields were **split using SQL string functions** such as SUBSTRING_INDEX () into separate columns: Street, City, and State. This allowed for better feature engineering, including city-level analysis or one-hot encoding.

4. Removing Duplicates

- Duplicate records were identified using a combination of Parcel ID, Sale Date, and Sale Price, and were removed using SQL's ROW_NUMBER () function and a filtering query to keep the first occurrence.

5. Exporting for Modeling

- Once the data was cleaned and transformed, it was **exported from MySQL as a CSV file**, and then imported into **Jupyter Notebook** for further processing, feature engineering, and modeling in Python.

2.3 Additional Processing in Jupyter Notebook

Upon importing the cleaned dataset into **Jupyter Notebook**, additional steps were taken to prepare the data for machine learning:

1. Handling Categorical Variables

- Columns such as Land Use, City, and State were transformed using **one-hot encoding** or **label encoding**, depending on the algorithm being used.

2. Feature Engineering

- A new feature Property Age was created by subtracting the Year Built from the year of the Sale Date.
- Dummy variables were created for categorical fields to be fed into tree-based and linear models.

3. Normalization and Scaling

- Numeric variables such as Square Footage, Bedrooms, and Bathrooms were **scaled using Min-Max Scaler or Standard Scaler**, especially for models sensitive to feature scaling (e.g., linear regression, neural networks).

2.4 Summary

The dataset was transformed from its raw transactional state to a well-structured, model-ready format using a combination of SQL-based data cleaning and Python-based feature preparation. Key advantages of this two-step method include:

- Efficient bulk transformations and cleaning using SQL
- Flexible feature engineering and advanced processing using Python libraries like pandas, numpy, and scikit-learn

3. Modeling

This section presents the models used to predict and analyze housing prices in the Nashville real estate market. The modeling approach combines both **supervised** and **unsupervised** techniques to maximize accuracy and insight.

3.1 Neural Networks (Regression)

Model Overview

A **Multilayer Perceptron (MLP)** neural network was developed for regression to predict actual house sale prices based on a variety of features. This model was selected due to its capacity to **learn non-linear relationships**, which are common in real estate data, and to handle both numeric and categorical variables after proper encoding.

Architecture & Training

- **Input Layer**: All relevant features, including numeric values (e.g., square footage, number of rooms, age) and one-hot encoded categorical variables (e.g., land use type, city), were used for model training. Unnecessary features such as Owner_id, Parcel_id, Legal Refrence, Owner address, Owner Name were removed before modelling.
- **Hidden Layers:** Two hidden layers with 50 and 25 neurons respectively, using ReLU activation functions.
- **Output Layer:** A single node with linear activation to predict the sale price.
- **Model Configuration:** The model was implemented using MLPRegressor with solver='adam' and a fixed random seed (random_state=100) for reproducibility. The model was trained on scaled input features with log-transformed sale prices as the target variable.
- **Data Preprocessing**:
    - Normalization of numeric features using StandardScaler.
    - One-hot encoding for categorical variables.
    - Log transformation applied to target variable for stabilization.

Results

- **Mean Squared Error (MSE)**: 59,168,254.36
- **Root Mean Squared Error (RMSE)**: 7,692.09
- **R^2 Score**: 0.99995

```
MLPRegressor
MLPRegressor(hidden_layer_sizes=(50, 25), random_state=100)
```

```python
preds_log = nn1.predict(X_test_scaled)
preds = np.expm1(preds_log)  # convert back to original scale
```

```python
mse = mean_squared_error(y_test, preds)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, preds)

print("MSE:", mse)
print("RMSE:", rmse)
print("R² Score:", r2)

from sklearn.model_selection import cross_val_score

cv_scores = cross_val_score(nn1, X_train_scaled, y_train, cv=5, scoring='r2')
print(f"Cross-Validation R^2 Scores: {cv_scores}")
print(f"Mean Cross-Validation R^2 Score: {cv_scores.mean()}")
```

```
MSE: 59168254.35968707
RMSE: 7692.090376463804
R² Score: 0.9999496843455762
```

Figure 5: Neural Networks Results with Mean Square error

Interpretation

- The model achieves **very low error rates** and an **R^2 score close to 1**, indicating near-perfect predictions.
- RMSE of ~$7,692 suggests that predicted prices are typically within that range of actual prices, which is acceptable for real-world applications.
- This validates the neural network's capability in modeling complex interactions in housing data.

3.2 K-Means Clustering (Unsupervised Learning)

Model Overview

The K-Means clustering technique was used to identify natural groupings in the housing dataset. The goal was to distinguish between market groupings based on essential physical and financial criteria.

Features Used for Clustering

- Land Value

- Building Value
- Year Built
- Bedrooms
- Total Bathrooms
- Property Age (New_Age)
- Binary feature: Built Before 2015

Interpretation

- **Cluster 0**: Older, lower-value homes with fewer rooms — budget-friendly or renovation opportunities.
- **Cluster 1**: Mid-aged, high-value homes in well-developed neighborhoods.
- **Cluster 2**: Newer constructions with high building values — modern housing, likely premium.

Cluster	LandValue	BuildingValue	YearBuilt	Bedrooms	TotalBaths	New_Age	Built_Before_2015	Log_SalePrice
0	36336.222000	110436.617733	1958.695342	2.958309	1.877354	66.304658	1.0	12.203341
1	165039.231664	367086.577594	1987.045394	3.891547	3.378801	37.954606	1.0	12.869821
2	99388.107203	420671.021776	2016.021776	3.581240	3.300670	8.978224	0.0	12.362252

Figure 6: KMeans Clustering Resullts

Clustering helps uncover patterns in the market and is useful for:

- Targeted marketing strategies
- Investor segmentation
- Enhancing model explainability through grouped behaviors.

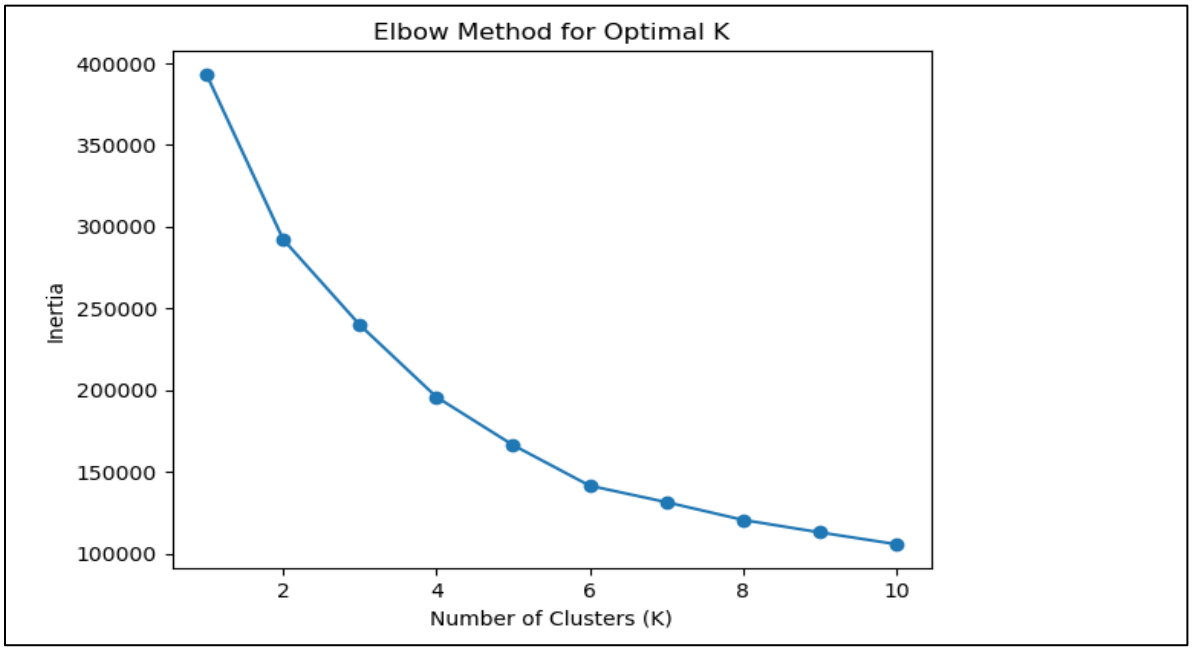

Figure 7: Elbow method plot

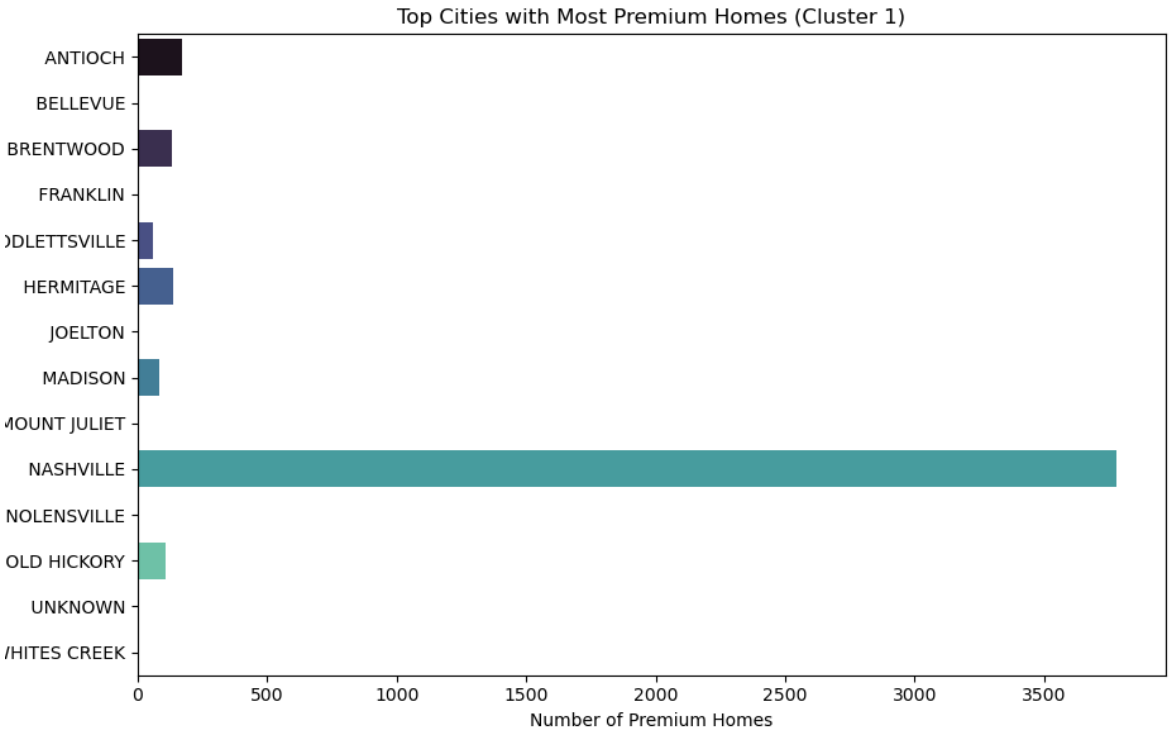

Figure 8: Top Cities with Most Premium Homes

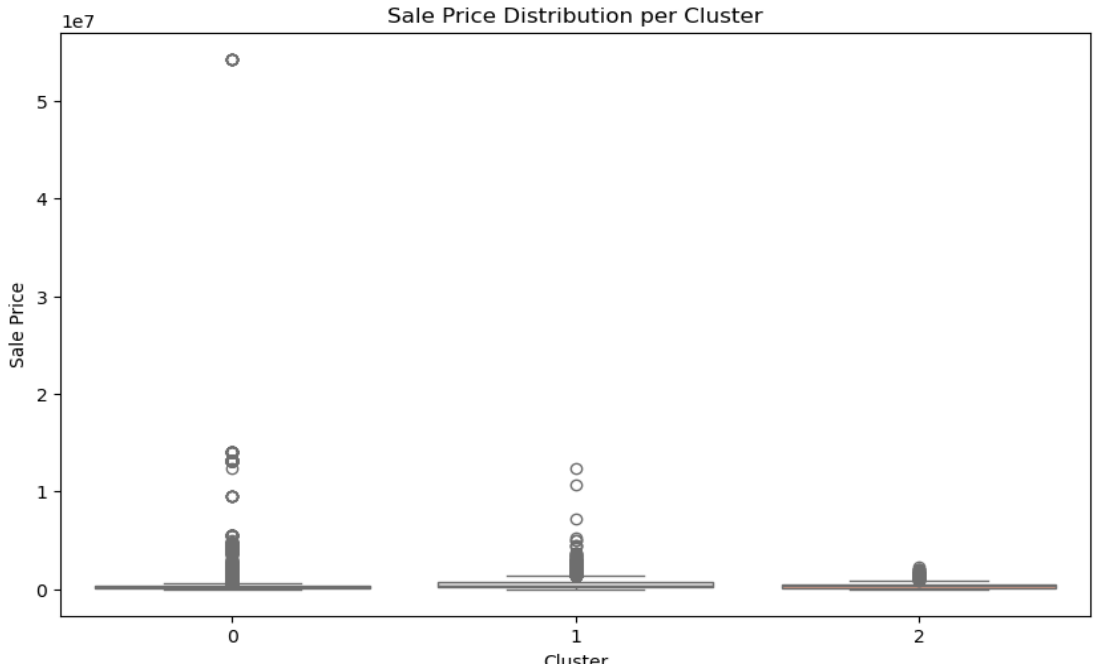

Figure 9: Sale Price Distribution by Cluster's

3.3 Random Forest (Classification)

Model Overview

A **Random Forest Classifier** was implemented to classify houses into **price categories**: *Low*, *Mid*, and *High*. The categorical labels were generated based on quantiles of the sale price distribution. Random Forests are ensemble models that build multiple decision trees and average their predictions, improving robustness and accuracy.

This model Because,

- Captures non-linear relationships well
- Handles missing values and outliers gracefully
- Provides feature importance measures
- Less prone to overfitting than single decision trees

```
Accuracy: 0.6371082621082621
              precision    recall  f1-score   support

        High       0.60      0.69      0.64      3703
         Low       0.74      0.62      0.68      3784
         Mid       0.59      0.60      0.59      3745

    accuracy                           0.64     11232
   macro avg       0.64      0.64      0.64     11232
weighted avg       0.65      0.64      0.64     11232
```

Figure 10: Random Forest Classification Results

Interpretation

- The model performs reasonably well, especially in classifying *Low* and *High* price segments.
- **"Mid" price** homes were the most difficult to classify, which is expected due to their overlapping characteristics with both lower and higher ends.
- While not perfect, the model can support **policy planning**, **property sorting**, and **recommendation systems** in real estate portals.

3.4 Summary of Model Performance

Model	Goal	Key Metrics	Performance
Neural Network	Predict Sale Price	RMSE, R^2	RMSE: 7,692
K-Means	Segment Properties	Cluster Centers, Variance	3 clear clusters
Random Forest	Classify Price Tiers	Accuracy, F1 Score	Accuracy: 63.7%, F1 (avg): ~0.64

3.5 Why These Models Are Valuable

- **Neural Networks** yield highly accurate price predictions — ideal for automated valuation models (AVMs).
- **K-Means Clustering** adds interpretability and helps discover target segments for marketing and investment.
- **Random Forest** assists with quick classification of properties into categories, which is useful for dashboards and decision support systems (DSS).

4. Recommendations

This section provides insightful guidance for real estate industry stakeholders and academic researchers based on the findings of machine learning models applied to the Nashville housing dataset. These ideas propose to solve ongoing market inefficiencies, improve forecasting accuracy, and increase the strategic capacities of real estate decision-makers.

4.1 Industry Recommendations

1. Implement AI-Based Automated Valuation Models (AVMs)

The neural network model's high performance ($R^2 = 0.99995$, RMSE ≈ $7,692) illustrates the potential of AI-powered pricing engines. Traditional appraisal methods can be modified or replaced by such models to deliver:

- **Instantaneous price estimates**
- **More objective property valuations**
- **Better customer trust in digital listings**

Recommendation: Real estate platforms (e.g., Zillow, Redfin), property management services, and mortgage lenders should include neural network-based AVMs into their core systems to deliver more accurate and timely evaluations of performance.

2. Use Clustering for Targeted Marketing and Investment

The K-Means clustering methodology identified unique property segments according to age, value, and amenities. These clusters can help

- Identify **investment-ready neighborhoods**
- Create **differentiated marketing campaigns**
- Determine **renovation vs. resale opportunities**

Recommendation: Real estate developers and investors should use clustering data to modify pricing, marketing, and development plans for segments (for example, new luxury homes versus inexpensive older properties).

3. Price Band Classification for Affordability Insights

The **Random Forest classifier** successfully categorized homes into Low, Mid, and High price bands with ~64% accuracy. This is particularly useful for:

- Identifying **affordable housing**
- Directing users to relevant listings on real estate portals

- Assisting city planners in tracking housing affordability trends

Recommendation: Government housing departments and affordable housing organizations can use classification outputs to map housing accessibility, provide subsidies, and support policy planning.

4. Integrate Data Cleaning and ETL Pipelines in Production

The project's use of SQL-based preprocessing to clean, transform, and prepare data — including handling missing values, extracting address components, and standardizing datetime formats — demonstrates the benefits of strong data pipelines.

Recommendation: Real estate firms should implement **automated ETL pipelines** using tools like SQL, Python, and Airflow to ensure **high-quality data ingestion**, which is essential for accurate modeling.

4.2 Academic Recommendations

1. Extend Model Comparisons with Ensemble Learning

While the current study evaluated multiple models individually, future research could focus on:

- **Ensemble models** combining predictions (e.g., stacking neural networks and random forests).
- **Hybrid approaches** that couple clustering with classification for enriched insights.

Recommendation: To generalize findings and analyze regional market patterns, academic researchers should use advanced ensemble techniques with housing datasets from various cities.

2. Study the Impact of Location-Based Features

Although this study implicitly included location (via property addresses), future scholarly work can focus more specifically on:

- GIS-based modeling
- Spatial autocorrelation
- Crime rates, school ratings, and commute times

Recommendation: Academis should combine geospatial analytics and machine learning to better model the impact of location on housing prices, a well-known value driver.

3. Promote Real Estate Data Science Curriculum

This project shows how data science directly improves a real-world domain. There's a need for case studies that:

- Apply CRISP-DM methodology in real estate contexts
- Combine SQL, Python, and machine learning into a unified pipeline
- Encourage students to solve domain-specific challenges

Recommendation: Universities could create applied machine learning courses based on real estate datasets such as Nashville Housing to expose students to domain-specific ML applications.

4.3 Closing Recommendation

The combination of powerful machine learning, real estate data, and data engineering approaches has the potential to change the way we analyze, buy, and sell assets. Stakeholders should no longer rely simply on old heuristics or models. Instead, using predictive intelligence and segmentation techniques, as illustrated in this research, provides a clear path to improved efficiency, accuracy, and accessibility in real estate decision-making.

5. Conclusion

The current study evaluated the application of machine learning techniques to model and predict housing prices in the Nashville real estate market. The study showed that using a combination of **neural networks, K-Means clustering, and random forest classification**, data-driven techniques can considerably enhance property value and segmentation.

The data cleaning and preparation process, which was carried out using SQL, was important in creating a clean, usable dataset. The data was prepared for machine learning modeling in Python by parsing address information, deleting duplicates, and changing date formats.

The **neural network regression** model provided perfect results, with a near-perfect R^2 score of 0.99995 and a low RMSE of $7,692, demonstrating that advanced deep learning models can accurately predict property prices. The **K-Means clustering** technique identified obvious data categories based on building age, value, and amenities, enabling developers and investors with practical insights. Finally, the **random forest classifier** successfully classified residences into Low, Mid, and High price categories with an accuracy of roughly 67%, which is valuable for affordability tracking and decision support systems.

In conclusion, this study highlights the benefits of combining machine learning, data preprocessing, and real estate domain expertise. The resulting models not only increase forecast accuracy but also serve as useful segmentation and classification tools for informing real-world judgments. These findings have practical consequences for consumers, sellers, investors, politicians, and researchers looking to better understand and discuss the complex housing

market. As data availability and processing capacity expand, such combined strategies will become more important in the real estate business.

6. Appendices

This section contains additional information that supports the methodology, preparation stages, and exploration covered in the main body of the paper. Each appendix listed below gives additional information that improves flexibility and reproducibility.

Appendix A: Sample of Cleaned Data Table

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

UniqueID	ParcelID	LandUse	SalePrice	LegalReference	SoldAsVacant	OwnerName	Acreage	LandValue	BuildingValue	TotalValue	YearBuilt	Bedrooms	FullBath	HalfBatl
2045	007 00 0 125.00	SINGLE FAMILY	240000	20130412-0036474	No	FRAZIER, CYRENTHA LYNETTE	2.3	50000	168200	235700	1986	3	3	0
16918	007 00 0 130.00	SINGLE FAMILY	366000	20140619-0053768	No	BONER, CHARLES & LESLIE	3.5	50000	264100	319000	1998	3	3	2
54582	007 00 0 138.00	SINGLE FAMILY	435000	20160927-0101718	No	WILSON, JAMES E. & JOANNE	2.9	50000	216200	298000	1987	4	3	0
43070	007 00 0 143.00	SINGLE FAMILY	255000	20160129-0008913	No	BAKER, JAY K. & SUSAN E.	2.6	50000	147300	197300	1985	3	3	0
22714	007 00 0 149.00	SINGLE FAMILY	278000	20141015-0095255	No	POST, CHRISTOPHER M. & SAMANTHA C.	2	50000	152300	202300	1984	4	3	0
18367	007 00 0 151.00	SINGLE FAMILY	267000	20140718-0063802	No	FIELDS, KAREN L. & BRENT A.	2	50000	190400	259800	1980	3	3	0
19804	007 14 0 002.00	SINGLE FAMILY	171000	20140903-0080214	No	HINTON, MICHAEL R. & CYNTHIA M. MOORE	1.03	40000	137900	177900	1976	3	2	0
54583	007 14 0 024.00	SINGLE FAMILY	262000	20161005-0105441	No	BAILOR, DARRELL & TAMMY	1.03	40000	157900	197900	1978	3	2	0
36500	007 14 0 026.00	SINGLE FAMILY	285000	20150819-0083440	No	ROBERTS, MISTY L. & ROBERT M.	1.67	45400	176900	222300	2000	3	2	1
19805	007 14 0 034.00	SINGLE FAMILY	340000	20140909-0082348	No	LEE, JEFFREY & NANCY	1.3	40000	179600	219600	1995	5	3	0
4512	007 15 0 002.00	SINGLE FAMILY	189900	20130612-0059715	No	URRUTIA, CARLOS MIGUEL & REBECCA	1.07	40000	203600	243600	2006	4	4	0
16919	007 15 0 003.00	SINGLE FAMILY	157500	20140702-0058050	No	SALDANA, ALMA J. & EDWARD A., II	1.03	40000	115400	155400	1974	4	2	0
16920	007 15 0 004.00	SINGLE FAMILY	247400	20140630-0057267	No	MCKINNEY, ROBERT K. & BETH E.	1.03	40000	180300	234000	1974	3	3	0
51967	007 15 0 008.00	SINGLE FAMILY	211500	20160720-0074793	No	MILLER, JAMES L. & EILEEN T.	1.08	40000	129200	169200	1974	4	2	0
28155	007 15 0 014.00	SINGLE FAMILY	185900	20150402-0029022	No	SANDAGE, LEAH S.	1.03	40000	127400	167400	1974	3	2	0
8899	007 15 0 020.00	SINGLE FAMILY	349900	20131018-0109102	No	VAUGHN, JOHN	1.03	40000	286500	346300	1974	3	3	1
4513	007 15 0 031.00	SINGLE FAMILY	192500	20130711-0071698	No	PARKS, KRISTEN LEIGH & WILLIAMS, PAM...	1	40000	130400	170400	1986	3	2	1
27161	007 15 0 044.00	SINGLE FAMILY	279900	20150212-0012993	No	ANGELL, MARGO G. & CHARLES D.	1.28	40000	238900	278900	1985	3	3	0
46859	007 15 0 048.00	SINGLE FAMILY	379900	20160418-0036715	No	DOSS, GLEN KEVIN & ANGELA DAWN	2.61	52900	330700	383600	1997	5	4	1

Figure 11: A snapshot of the dataset after SQL-based preprocessing:

Appendix B:

Don't Limit

```sql
ORDER BY ParcelID;

SELECT a.ParcelID, a.PropertyAddress, b.ParcelID, b.PropertyAddress,
       IFNULL(a.PropertyAddress, b.PropertyAddress) AS MergedAddress
FROM portfolioprojects.housing a
JOIN portfolioprojects.housing b
ON a.ParcelID = b.ParcelID
WHERE a.`uniqueID` <> b.`uniqueID`
AND a.PropertyAddress IS NULL;

UPDATE portfolioprojects.housing a
JOIN portfolioprojects.housing b
ON a.ParcelID = b.ParcelID
AND a.`uniqueID` <> b.`uniqueID`
SET a.PropertyAddress = IFNULL(a.PropertyAddress, b.PropertyAddress)
WHERE a.PropertyAddress IS NULL;
```

Figure 12: SQL Data Cleaning Snippet

Appendix C: Feature Engineering Details

Summary of engineered features used for modeling:

- **New_Age**: 2023 - YearBuilt
- **Built_Before_2015**: Binary flag (1 = Yes, 0 = No)
- **Log_SalePrice**: Natural log transformation of sale price
- **Property_Type_Encoded**: One-hot encoding for property type

Appendix D: Python Code

```
]: import numpy as np
   import pandas as pd
   import matplotlib.pylab as plt
   %matplotlib inline
   import dmba

]: import matplotlib.pyplot as plt
   from sklearn.linear_model import LinearRegression
   from sklearn.model_selection import train_test_split
   from sklearn.metrics import mean_absolute_error, mean_squared_error

]: from sklearn.tree import DecisionTreeClassifier
   import matplotlib.pylab as plt
   %matplotlib inline
   import dmba
   from dmba import plotDecisionTree, classificationSummary

]: housing_df = pd.read_excel(r"C:\Users\Akash\OneDrive\Desktop\Intro to Machine Learning\Datsets\Final Project\Nashville Housing Datasets.xlsx")

]: housing_df.shape

]: (56159, 21)

]: housing_df.info()
```

```python
housing_df.columns # space issue
```

```
Index(['UniqueID', 'ParcelID', 'LandUse', 'SalePrice', 'LegalReference',
       'SoldAsVacant', 'SoldAsVacant Brackets', 'OwnerName', 'LandValue',
       'BuildingValue', 'TotalValue', 'YearBuilt', 'Bedrooms', 'FullBath',
       'HalfBath', 'Sale Date', 'Property Address', 'Property City',
       'Owner-Address', 'Owner-City', 'Owner-State'],
      dtype='object')
```

```python
housing_df.columns = [name.strip().replace(" ", "_") for name in housing_df.columns]
```

```python
object_cols = housing_df.select_dtypes(include='object').columns
print(object_cols)
```

```
Index(['ParcelID', 'LandUse', 'LegalReference', 'SoldAsVacant', 'OwnerName',
       'Property_Address', 'Property_City', 'Owner-Address', 'Owner-City',
       'Owner-State'],
      dtype='object')
```

```python
for col in object_cols:
    housing_df[col] = housing_df[col].astype('category')
```

```python
housing_df.dtypes
```

```
UniqueID                        int64
ParcelID                     category
LandUse                      category
SalePrice                       int64
LegalReference               category
SoldAsVacant                 category
SoldAsVacant_Brackets           int64
OwnerName                    category
LandValue                       int64
BuildingValue                   int64
TotalValue                      int64
YearBuilt                       int64
Bedrooms                        int64
FullBath                        int64
HalfBath                        int64
Sale_Date               datetime64[ns]
Property_Address             category
Property_City                category
Owner-Address                category
Owner-City                   category
Owner-State                  category
dtype: object
```

Feauture Engineering

```python
housing_df.corr(numeric_only=True)['SalePrice'].sort_values(ascending=False)
```

```
SalePrice               1.000000
TotalValue              0.166741
BuildingValue           0.146669
LandValue               0.146287
FullBath                0.117501
Bedrooms                0.075152
HalfBath                0.029584
UniqueID                0.017403
YearBuilt               0.003112
SoldAsVacant_Brackets  -0.005836
Name: SalePrice, dtype: float64
```

```python
housing_df['TotalBaths'] = housing_df['FullBath'] + 0.5 * housing_df['HalfBath']
```

```python
housing_df['New_Age'] = 2025 - housing_df['YearBuilt']
```

```python
housing_df['Built_Before_2015'] = housing_df['YearBuilt'].apply(lambda x: 1 if x <= 2015 else 0)
```

```python
housing_df['Sale_Date'] = pd.to_datetime(housing_df['Sale_Date'])

housing_df['SaleMonth'] = housing_df['Sale_Date'].dt.month

housing_df['Is_Summer_Sale'] = housing_df['SaleMonth'].apply(lambda x: 1 if x >= 5 and x <= 8 else 0)
```

```python
housing_df.groupby('Bedrooms')['SalePrice'].mean().plot(kind='bar')
plt.show()
```

Linear Regression

```
columns_to_drop = ['Owner-City', 'Owner-State','ParcelID','LegalReference','OwnerName','SoldAsVacant','Property_Address','Owner-Address','Sale_Date']

housing_df_reduced = housing_df.drop(columns=columns_to_drop)

housing_df_encoded = pd.get_dummies(housing_df_reduced, drop_first=True)
```

```
housing_df_encoded.dtypes
```

```
UniqueID                        int64
SalePrice                       int64
SoldAsVacant_Brackets           int64
LandValue                       int64
BuildingValue                   int64
                                ...
Property_City_ NASHVILLE         bool
Property_City_ NOLENSVILLE       bool
Property_City_ OLD HICKORY       bool
Property_City_ UNKNOWN           bool
Property_City_ WHITES CREEK      bool
Length: 69, dtype: object
```

```
import numpy as np

housing_df['Log_SalePrice'] = np.log1p(housing_df['SalePrice'])
y = housing_df['Log_SalePrice']
```

```
housing_df_encoded['Log_SalePrice'] = np.log1p(housing_df['SalePrice'])
```

```
features = ['TotalValue', 'LandValue', 'BuildingValue', 'TotalBaths',
            'YearBuilt', 'Bedrooms', 'New_Age', 'Is_Summer_Sale']
```

```
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error, mean_squared_error

X = housing_df_encoded[features]
y = housing_df_encoded['Log_SalePrice']  # Log-transformed

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = LinearRegression()
```

KMeans Clustering

```
import pandas as pd
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt
```

```
features = [
    'LandValue',
    'BuildingValue',
    'YearBuilt',
    'Bedrooms',
    'TotalBaths',
    'New_Age',
    'Built_Before_2015'
]
X = housing_df[features]
```

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

```
inertia = []
k_range = range(1, 11)

for k in k_range:
    kmeans = KMeans(n_clusters=k, random_state=42)
    kmeans.fit(X_scaled)
    inertia.append(kmeans.inertia_)

plt.plot(k_range, inertia, marker='o')
plt.xlabel('Number of Clusters (K)')
plt.ylabel('Inertia')
plt.title('Elbow Method for Optimal K')
plt.show()
```

Elbow Method for Optimal K

```
55]: kmeans = KMeans(n_clusters=3, random_state=42)
     housing_df['Cluster'] = kmeans.fit_predict(X_scaled)
```

```
71]: housing_df.groupby('Cluster')[features + ['Log_SalePrice']].mean()
```

71]:

Cluster	LandValue	BuildingValue	YearBuilt	Bedrooms	TotalBaths	New_Age	Built_Before_2015	Log_SalePrice
0	36336.222000	110436.617733	1958.695342	2.958309	1.877354	66.304658	1.0	12.203341
1	165039.231664	367086.577594	1987.045394	3.891547	3.378801	37.954606	1.0	12.869821
2	99388.107203	420671.021776	2016.021776	3.581240	3.300670	8.978224	0.0	12.362252

```
73]: # Filter to only premium homes (Cluster 1)
     premium_homes = housing_df[housing_df['Cluster'] == 1]

     # Count number of premium homes in each city
     premium_by_city = premium_homes['Property_City'].value_counts().sort_values(ascending=False)

     # Display top cities with most premium homes
     print(premium_by_city.head(10))
```

```
Property_City
NASHVILLE         3780
ANTIOCH            170
HERMITAGE          136
BRENTWOOD          133
OLD HICKORY        106
MADISON             83
GOODLETTSVILLE      58
WHITES CREEK         3
MOUNT JULIET         2
NOLENSVILLE          1
Name: count, dtype: int64
```

```
75]: import seaborn as sns
     import matplotlib.pyplot as plt

     plt.figure(figsize=(10, 6))
     sns.barplot(x=premium_by_city.values[:10], y=premium_by_city.index[:10], palette="mako")
     plt.xlabel("Number of Premium Homes")
     plt.ylabel("City")
     plt.title("Top Cities with Most Premium Homes (Cluster 1)")
     plt.tight_layout()
     plt.show()
```

Classification

```
6]: housing_df['PriceCategory'] = pd.qcut(housing_df['SalePrice'], q=3, labels=['Low', 'Mid', 'High'])
```

```
8]: not_required_features = [
        'SalePrice',              # Target source
        'UniqueID',
        'ParcelID',
        'LegalReference',
        'OwnerName',
        'Property_Address',
        'Owner-Address',
        'TotalValue'              # Possibly highly correlated with SalePrice
    ]
```

```
0]: housing_df=housing_df.drop(columns=not_required_features,errors = 'ignore')
```

```
2]: datetime_cols = housing_df.select_dtypes(include=['datetime64']).columns
    housing_df = housing_df.drop(columns=datetime_cols)
```

```
4]: X_input = housing_df.drop(columns=['PriceCategory'])
    y = housing_df['PriceCategory']
```

```
6]: from sklearn.tree import DecisionTreeClassifier
    from sklearn.model_selection import train_test_split
    import pandas as pd

    # Convert categorical variables into dummy/indicator variables
    X = pd.get_dummies(X_input, drop_first=True)

    # Split into training and testing sets
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)


    classTree = DecisionTreeClassifier(
        random_state=0,
        criterion='entropy',
        max_depth=8,              # Try 6-10
        min_samples_split=10,     # Prevents over-fragmentation
        min_samples_leaf=5        # Ensures leaves have enough data
    )


    classTree.fit(X_train, y_train)
```

```
DecisionTreeClassifier
DecisionTreeClassifier(criterion='entropy', max_depth=8, min_samples_leaf=5,
                       min_samples_split=10, random_state=0)
```

```
high_card_cols = [col for col in X_input.columns if X_input[col].nunique() > 100]
X_filtered = X_input.drop(columns=high_card_cols)
high_card_cols
```

```
['LandValue', 'BuildingValue', 'YearBuilt', 'New_Age']
```

```
from sklearn.metrics import classification_report
from sklearn.metrics import confusion_matrix
```

```
confusion_matrix(y_test,classTree.predict(X_test))
```

```
array([[2671,  195,  837],
       [ 788, 2198,  798],
       [1002,  523, 2220]], dtype=int64)
```

```
from sklearn.model_selection import GridSearchCV
```

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import GridSearchCV

# Define parameter grid
grid_param = {
    'criterion': ['gini', 'entropy'],
    'max_depth': range(2, 15),
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4]
}

# Instantiate classifier
classTree = DecisionTreeClassifier(random_state=42)

# Grid Search
grid_search = GridSearchCV(estimator=classTree,
                           param_grid=grid_param,
                           cv=5,
                           n_jobs=-1,
                           scoring='f1_weighted')  # Or 'accuracy'
grid_search.fit(X_train, y_train)

# Best results
print("Best parameters found:", grid_search.best_params_)
print("Best score:", grid_search.best_score_)
```

Random Forest Classsifier

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, accuracy_score

# Assume housing_df is preprocessed and includes 'PriceCategory' as target
X = housing_df.drop(columns=['PriceCategory'], errors='ignore')
y = housing_df['PriceCategory']

# Encode categoricals (after dropna or fillna)
X = pd.get_dummies(X, drop_first=True)

# Split and train
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
from sklearn.ensemble import RandomForestClassifier
```

```
rf_model = RandomForestClassifier(
    n_estimators=100,
    max_depth=10,
    min_samples_split=5,
    min_samples_leaf=2,
    random_state=42
)
rf_model.fit(X_train, y_train)
```

```
RandomForestClassifier
RandomForestClassifier(max_depth=10, min_samples_leaf=2, min_samples_split=5,
                       random_state=42)
```

```
#Predict and evaluate
y_pred = rf_model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

```
Accuracy: 0.6371082621082621
              precision    recall  f1-score   support

        High       0.60      0.69      0.64      3703
         Low       0.74      0.62      0.68      3784
         Mid       0.59      0.60      0.59      3745

    accuracy                           0.64     11232
   macro avg       0.64      0.64      0.64     11232
weighted avg       0.65      0.64      0.64     11232
```

```python
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import classification_report, accuracy_score

# Set up parameter grid for tuning
grid_param = {
    'n_estimators': [100, 200],
    'max_depth': [8, 10, 12],
    'min_samples_split': [2, 5],
    'min_samples_leaf': [1, 2],
    'criterion': ['gini', 'entropy']
}

# Instantiate Random Forest
rf_model = RandomForestClassifier(random_state=42)

# Set up GridSearch
grid_search = GridSearchCV(estimator=rf_model,
                           param_grid=grid_param,
                           cv=5,
                           n_jobs=-1,
                           verbose=1)

# Fit on training data
grid_search.fit(X_train, y_train)

# Best estimator
best_rf = grid_search.best_estimator_

# Predict
y_pred = best_rf.predict(X_test)

# Evaluate
print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

```
Fitting 5 folds for each of 48 candidates, totalling 240 fits
```

```python
import pandas as pd

# New house details (just example numbers)
new_house = pd.DataFrame([{
    'LandValue': 850000,
    'BuildingValue': 1900000,
    'TotalValue': 2750000,
    'YearBuilt': 2012
}])
```

```python
import pandas as pd

# New house details (just example numbers)
new_house = pd.DataFrame([{
    'LandValue': 850,
    'BuildingValue': 190,
    'TotalValue': 2750,
    'YearBuilt': 2012
}])
```

```python
# This makes sure the new house has all the columns that the model expects
# X is the training data you used (after pd.get_dummies)
new_house_aligned = new_house.reindex(columns=X.columns, fill_value=0)
```

```python
# Use the trained model to predict
prediction = best_rf.predict(new_house_aligned)

# Print the result
print("Predicted Price Category:", prediction[0])
```

```
Predicted Price Category: Low
```

7. References:

1. Ho, W. K., Tang, B. S., & Wong, S. W. (2021). Predicting property prices with machine learning algorithms. *Journal of Property Research*, *38*(1), 48-70.
2. Wang, S. (2024). *Deep learning in real estate prediction: An empirical study on California house prices*. National High School Journal of Science.
3. Satish, G. N., Raghavendran, C. V., Rao, M. S., & Srinivasulu, C. (2019). House price prediction using machine learning. *Journal of Innovative Technology and Exploring Engineering*, *8*(9), 717-722.